
Temporal Reliability Token Ordering for Reasoning in Diffusion Language Models

Korea University COSE461 Final Project

Jaeyoon Jung

Department of Electrical Engineering
Team 6
2021171077

Byungmin Park

Department of Electrical Engineering
Team 6
2021170933

Eunseo Choi

Department of Electrical Engineering
Team 6
2023171069

Abstract

Diffusion language models (DLMs) generate text through iterative denoising, giving them the flexibility to choose which masked token positions to reveal during inference. However, many decoding policies still rely on instantaneous confidence scores such as top-1 probability, which may not capture whether a prediction is reliable across denoising steps. In this paper, we study temporal reliability as a token-ordering signal for deterministic masked diffusion decoding. Building on probability margin, the gap between the top-1 and top-2 token probabilities, we propose two training-free policies that modify only the token-position ranking rule. GTTO (Gated Temporal Token Ordering) combines margin with top-1 temporal stability and activates the stability term only when the current prediction is locally ambiguous. MargExKL (Margin with Exponential KL Penalty) instead penalizes margin using step-to-step KL divergence, discounting positions whose predictive distributions shift sharply over time. We evaluate LLaDA-8B-Instruct and LLaDA-1.5 on GSM8K, MATH500, SVAMP, and Countdown under a controlled deterministic setup. At the main decoding setting, both policies improve over top1_prob across all four reasoning benchmarks, with the largest gains on Countdown and MATH500. The two methods show task-dependent differences between token-level and distribution-level stability. These findings suggest that temporal information in DLMs can guide not only answer-level aggregation or early commitment, but also token-level decisions that shape the decoding trajectory.

1 Introduction

Inference-time generation strategies play an important role in reasoning with language models. Chain-of-thought prompting encourages models to produce intermediate reasoning steps [1], while self-consistency improves reliability by aggregating multiple sampled reasoning paths [2]. These results suggest that reasoning performance depends not only on model parameters, but also on how intermediate decisions are generated and selected.

Diffusion language models (DLMs) offer a different generation paradigm from autoregressive language models. Instead of producing tokens strictly from left to right, DLMs begin from masked sequences and iteratively denoise them, predicting multiple masked tokens in parallel and committing

only a subset of positions at each step. This generation process makes token ordering an explicit inference-time decision: the model must decide which masked positions to reveal early and which positions to leave for later refinement.

Token ordering is important because each committed token becomes part of the context for subsequent denoising steps. A reliable early commitment can help later predictions, while an unstable or premature commitment can steer the remaining trajectory in a worse direction. However, many decoding policies still use simple local confidence scores, most commonly the top-1 token probability, to choose which positions to reveal. This can be insufficient because a high top-1 probability does not necessarily imply a decisive prediction if the second-best token has a similar probability. Kim et al. [3] showed that probability margin—the gap between the top-1 and top-2 predicted probabilities—is a more reliable token-ordering signal than raw maximum probability, a finding that directly motivates our use of probability margin as the base ranking signal.

However, probability margin is still a snapshot-level measure: it only describes the current predictive distribution. Recent work suggests that diffusion trajectories contain useful temporal information. Wang et al. [4] observed that correct intermediate answers can appear and later disappear during diffusion decoding, motivating Temporal Self-Consistency Voting. Li et al. [5] used confidence gaps to decide when a partially refined answer can be committed early. Related decoding methods also suggest that cross-step consistency can complement instantaneous confidence [6, 7]. These observations motivate our question: can temporal reliability guide token ordering itself, rather than being used only for post-hoc aggregation or commitment?

We propose two training-free token-ordering policies based on temporal reliability. GTTO combines probability margin with top-1 temporal stability, activating the stability term only when the current prediction is locally ambiguous. Thus, confident positions remain governed by margin, while ambiguous positions can be guided by recent token stability. MargExKL instead uses step-to-step KL divergence to measure distribution-level instability and discounts positions whose predictive distributions shift sharply across denoising steps. Together, the two policies capture complementary notions of temporal reliability: token-level stability and distribution-level stability.

We evaluate these policies with LLaDA-8B-Instruct and LLaDA-1.5 on GSM8K, MATH500, SVAMP, and Countdown, covering arithmetic word problems, competition-style mathematics, and combinatorial arithmetic reasoning. Our results show that temporal reliability improves over standard top-1 probability ordering across reasoning benchmarks. We further compare against probability-margin ordering and analyze generation-length sensitivity to examine when temporal signals help or fail.

Our main contributions are as follows:

- We study temporal reliability as a token-ordering signal in deterministic masked diffusion decoding.
- We propose GTTO, a gated temporal-stability policy that combines probability margin with top-1 token stability under local ambiguity.
- We propose MargExKL, a KL-penalized margin policy that uses step-to-step distributional change as a complementary reliability signal.
- We evaluate these policies on reasoning benchmarks under a controlled setup, comparing them against standard top-1 probability ordering and probability-margin ordering.

2 Related Work

2.1 Diffusion Large Language Models

Diffusion language models generate text by iteratively denoising masked token sequences, offering an alternative to left-to-right autoregressive generation [8, 9, 10, 11]. Large-scale models such as LLaDA [12] and Dream [13] demonstrate competitive performance in the 7–8B parameter regime and serve as the experimental platforms for our token-ordering policies.

2.2 Temporal Dynamics and Self-Consistency in Diffusion Decoding

A key observation motivating recent inference improvements is that intermediate predictions at each denoising step carry meaningful signal that standard decoding discards. Wang et al. [4] coined the

term *temporal oscillation* to describe correct answers emerging at intermediate steps only to be overwritten later, and proposed Temporal Self-Consistency Voting via weighted majority vote across steps. Chen et al. [6] pursued a related direction using cross-step consistency as both a selection criterion and a proxy for sampling error. Our methods draw on this temporal-stability insight, but apply it directly to token-position ranking during decoding.

2.3 Adaptive Inference and Probability Margin in Masked Diffusion Models

The observation that token decoding order substantially affects output quality has motivated several adaptive inference strategies. Kim et al. [3] showed that probability margin—the gap between the top-1 and top-2 predicted probabilities—is a more reliable token-ordering signal than raw maximum probability, a finding that directly motivates our use of probability margin as the base ranking signal.

2.4 Distribution-Level Stability Signals

Kim et al. [7] introduced KL-Adaptive Stability Sampling (KLASS), which tracks KL divergence between consecutive-step predictive distributions and delays unmasking until a token is both confident and stable, achieving up to $2.78\times$ wall-clock speedup on reasoning tasks. Our MargExKL policy adapts this distribution-level stability idea to token ordering, using step-to-step KL divergence as a per-position discount on the probability margin rather than as a global commitment gate.

3 Approach

We study whether temporal reliability—signals derived from how token predictions evolve across denoising steps—can improve token-ordering decisions in masked diffusion decoding. Our starting point is the probability margin baseline of Kim et al. [3], which we treat as the primary improved baseline throughout. We then propose two extensions that incorporate temporal signals into the ranking score: GTTO, which uses discrete top-1 token stability under a gating condition, and MargExKL, which uses continuous distributional stability via step-to-step KL divergence. Table 1 summarizes all methods compared in this work.

Method	Ranking score	Temporal use
top1_prob	$p_{\text{top1}}(i, t)$	None
prob_margin	$p_{\text{top1}}(i, t) - p_{\text{top2}}(i, t)$	None
temporal_margin	$m_i(t) + \lambda s_i(t)$	Always
GTTO	$m_i(t) + \lambda s_i(t) \mathbf{1}[m_i(t) < \tau]$	Gated
MargExKL	$m_i(t) \cdot \exp(-\gamma \cdot \text{KL}_i(t))$	Always

Table 1: Token-position ranking policies compared in this work. Here, $m_i(t) = p_{\text{top1}}(i, t) - p_{\text{top2}}(i, t)$ denotes probability margin, $s_i(t)$ denotes temporal stability, λ controls the temporal stability weight, τ is the ambiguity threshold, and γ controls the KL penalization strength.

3.1 Preliminaries: Semi-Autoregressive Block Decoding

We build on LLaDA [12], which generates text by iteratively denoising a fully masked sequence. At inference time, the output sequence of length L is partitioned into contiguous blocks of size B , processed left-to-right. Within each block, the model repeatedly predicts masked tokens in parallel and commits a subset of positions before continuing refinement.

At each denoising step t within the current block, the model computes a predictive distribution over all currently masked positions, conditioned on previously committed tokens. A *token-ordering policy* ranks masked positions by a scalar score $\phi_i(t)$ and unmaskes the top-ranked positions according to the transfer schedule. Newly committed tokens immediately enter the conditioning context, so different token-ordering policies can lead to different intermediate distributions and reasoning trajectories.

Unless otherwise stated, we use block length $B = 32$, transfer two tokens per step, and temperature $= 0$ greedy decoding. Thus each block requires $S = 16$ denoising steps, and the total diffusion-step

budget is $D = (L/B)S = L/2$. For $L \in \{128, 256, 512\}$, this gives $D \in \{64, 128, 256\}$. We use exp-weighted Temporal Self-Consistency Voting (TSCV) [4] as the aggregation mechanism.

3.2 Notation

For a masked position i at denoising step t within the current block, we define:

$$m_i(t) = p_{\text{top1}}(i, t) - p_{\text{top2}}(i, t) \quad (\text{probability margin}), \quad (1)$$

$$s_i(t) = \frac{r_i(t)}{t} \quad (\text{block-normalized run-length stability}), \quad (2)$$

$$\text{KL}_i(t) = \sum_{k=1}^M p_i^{(t)}(v_k) \log \frac{p_i^{(t)}(v_k)}{p_i^{(t-1)}(v_k)} \quad (\text{truncated step-to-step KL proxy}), \quad (3)$$

where $p_{\text{top1}}(i, t)$ and $p_{\text{top2}}(i, t)$ are the highest and second-highest softmax probabilities at position i and step t . The run length $r_i(t)$ counts consecutive steps ending at t for which position i maintains the same top-1 prediction:

$$r_i(t) = \begin{cases} r_i(t-1) + 1 & \text{if } \hat{x}_i^{(t)} = \hat{x}_i^{(t-1)}, \\ 1 & \text{otherwise,} \end{cases} \quad (4)$$

so that $s_i(t) \in (0, 1]$ measures how continuously the current top-1 prediction has persisted within the block. For $\text{KL}_i(t)$, $\{v_k\}_{k=1}^M$ denotes the top- M tokens at step t with $M = 10$, and probabilities are clamped from below by $\epsilon = 10^{-8}$ for numerical stability.

3.3 Token-Ordering Policies

We study five policies, progressing from snapshot-based baselines to temporally-informed methods. The first two policies (`top1_prob` and `prob_margin`) serve as baselines; the remaining three (`temporal_margin`, `GTT0`, and `MargExKL`) are original contributions of this work. All policies are implemented as drop-in replacements for the default unmasking rule within LLaDA’s semi-AR decoding loop, extending the TIAF codebase [4], which we use under its open-source license. The TSCV aggregation was already implemented therein; the margin computation, temporal stability tracking, and KL penalty were coded by us.

top1_prob (baseline).

$$\phi_i(t) = p_{\text{top1}}(i, t). \quad (5)$$

The standard rule: unmask positions with the highest top-1 softmax probability first. Simple and computationally free, but susceptible to softmax miscalibration — a high p_{top1} does not distinguish a decisive prediction (e.g., 0.85 vs. 0.10) from an ambiguous one (e.g., 0.90 vs. 0.88).

prob_margin (baseline).

$$\phi_i(t) = m_i(t). \quad (6)$$

Proposed by Kim et al. [3], margin replaces absolute probability with *relative decisiveness*. Because it measures separation between the top two candidates rather than magnitude alone, it provides a stronger local ambiguity signal than raw maximum probability. In our experiments, `prob_margin` is a competitive baseline and improves over `top1_prob` on several tasks, but it does not uniformly dominate it.

temporal_margin: Unconditional Temporal Stability (original).

$$\phi_i(t) = m_i(t) + \lambda \cdot s_i(t). \quad (7)$$

A direct way to use temporal information is to add token-level stability $s_i(t)$ to the probability margin. This rule ranks positions higher when their current top-1 prediction has persisted across recent denoising steps. However, this unconditional use of temporal stability can be harmful: when the current margin is already decisive, the stability term may unnecessarily re-rank otherwise reliable choices. As shown in Appendix B, this naive rule performs worse than `prob_margin` in a GSM8K smoke test, motivating the gated design below.

GTTO: Gated Temporal Token Ordering (original).

$$\phi_i(t) = m_i(t) + \lambda \cdot s_i(t) \cdot \mathbf{1}[m_i(t) < \tau]. \quad (8)$$

GTTO activates the temporal stability term only when the current margin falls below an ambiguity threshold τ . For unambiguous positions ($m_i(t) \geq \tau$), GTTO reduces exactly to `prob_margin`. For ambiguous positions ($m_i(t) < \tau$), it uses recent top-1 stability as an additional reliability signal. This gated design lets temporal information guide difficult token-ordering decisions without perturbing already confident margin-based choices. We evaluate $(\lambda, \tau) \in \{(0.05, 0.15), (0.05, 0.12), (0.10, 0.15)\}$ as competitive GTTO configurations and report their cross-task behavior in Appendix C.

MargExKL: KL-Penalized Margin (original).

$$\phi_i(t) = m_i(t) \cdot \exp(-\gamma \cdot \text{KL}_i(t)). \quad (9)$$

MargExKL uses step-to-step distributional change as a complementary temporal reliability signal. Instead of tracking only top-1 token agreement, it penalizes probability margin using a truncated KL proxy between consecutive predictive distributions. Positions whose predictive distributions shift sharply from the previous step receive a discounted margin score, even if their current margin is high. As $\gamma \rightarrow 0$, the policy recovers pure `prob_margin`; larger γ applies stronger instability penalization. We evaluate $\gamma \in \{0.3, 0.5\}$. Compared to GTTO, MargExKL provides a continuous distribution-level counterpart to top-1 stability.

3.4 Implementation

All five policies are implemented as drop-in replacements for the default unmasking rule within LLaDA’s semi-AR decoding loop, building on the TIAF codebase [4]. Margin computation requires only the top-2 softmax probabilities and adds negligible overhead. Temporal stability maintains a run-length counter for the top-1 token at each position, which costs $O(B)$ per block step. For MargExKL, we compute a truncated top- M KL proxy with $M = 10$. Although the score is evaluated over the top- M tokens, our implementation retrieves previous-step probabilities from the stored predictive distribution, leading to an $O(B \times |\mathcal{V}|)$ memory overhead per block. No policy modifies model weights, prompts, or the answer parser; intervention occurs solely at the token-position ranking stage of the denoising loop.

4 Experiments**4.1 Data**

We evaluate our token-ordering policies on four reasoning benchmarks, following the experimental setup of Wang et al. [4]. These cover arithmetic word problems (GSM8K, SVAMP), competition-style mathematics (MATH500), and combinatorial arithmetic reasoning (Countdown).

GSM8K is a dataset of approximately 8,500 grade school math word problems requiring multi-step arithmetic reasoning, of which 1,319 are used for evaluation.

MATH500 is a curated 500-problem subset of the MATH benchmark, covering diverse competition-level topics.

SVAMP consists of elementary math word problems designed to test robustness to structural variations in problem phrasing.

Countdown is a combinatorial arithmetic task in which the model must reach a target number using basic arithmetic operations on a given set of numbers.

4.2 Evaluation Method

We use vote-answer accuracy as the primary evaluation metric. Candidate answers parsed from intermediate denoising steps are aggregated using exponentially weighted Temporal Self-Consistency Voting (TSCV) [4], where each step’s prediction is weighted by $e^{\alpha \cdot t}$, and we set $\alpha = 5.0$ in all experiments. The most-voted answer is taken as the final prediction.

Model	Method	GSM8K	MATH500	SVAMP	Countdown
LLaDA-8B-Instruct	top1_prob	69.52	27.00	85.00	16.02
	prob_margin	69.07	27.80	87.00	19.92
	GTTO [†]	71.11	29.20	87.33	22.66
		+1.59	+2.20	+2.33	+6.64
	MargExKL [‡]	70.81	31.40	86.33	23.05
		+1.29	+4.40	+1.33	+7.03
LLaDA-1.5	top1_prob	71.11	27.20	85.67	18.75
	prob_margin	72.63	28.00	87.33	22.27
	GTTO [†]	74.07	29.60	88.67	23.83
		+2.96	+2.40	+3.00	+5.08
	MargExKL [‡]	73.39	31.60	87.67	25.39
		+2.28	+4.40	+2.00	+6.64

Table 2: Main results (vote-answer accuracy, gen_length = 128). All gains are measured against top1_prob. [†] GTTO and [‡] MargExKL use tuned hyperparameters; additional sweep and cross-task results are reported in Appendix C and D.

Because TSCV pools information across the full denoising trajectory, it reduces sensitivity to transient oscillations in individual steps and provides a stable signal for comparing token-ordering policies. All accuracy values are reported as percentages of correctly answered problems, using exact-match string comparison after standard answer normalization.

4.3 Experimental Details

We evaluate on two model checkpoints, LLaDA-8B-Instruct [12] and LLaDA-1.5, without any fine-tuning or parameter modification. The generation hyperparameters shared across all conditions are described in Section 3.1. We report main results at $L = 128$; generation-length sensitivity ($L \in \{256, 512\}$) is analyzed in Section 5, with additional hyperparameter comparisons and ablation results in Appendices C–F.

4.4 Results

At the main setting $L = 128$, both methods improve over top1_prob across all four benchmarks and both model checkpoints, with the largest gains on Countdown and MATH500; full per-task numbers are reported in Table 2.

The probability-margin baseline is substantially stronger than raw top-1 confidence on several tasks, confirming that local ambiguity is an important token-ordering signal. Nevertheless, the proposed temporal reliability policies usually improve further over prob_margin. GTTO improves over prob_margin on all four tasks for both model checkpoints. MargExKL also improves over prob_margin on most settings, with the main exception being SVAMP under LLaDA-8B-Instruct, where it slightly underperforms prob_margin. This suggests that temporal reliability is best viewed as a complement to margin-based ordering rather than a replacement for it.

The two proposed methods show different strengths. GTTO is more competitive on GSM8K and SVAMP, where top-1 token stability appears to be a useful reliability signal. MargExKL is stronger on MATH500 and Countdown, suggesting that distribution-level stability is more helpful on tasks with more diverse or less locally stable reasoning trajectories. We analyze this task-dependent behavior and generation-length sensitivity in Section 5.

5 Analysis

We analyze four aspects of the results: the task-dependent split between GTTO and MargExKL, the role of probability margin, the effect of generation length, and the failure modes revealed by the temporal-margin and exp-KL ablations.

Model	Method	GSM8K			MATH500			SVAMP			Countdown		
		128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	top1_prob	69.52	78.39	79.53	27.00	33.20	35.00	85.00	84.67	85.33	16.02	16.80	16.41
	prob_margin	69.07	77.26	78.77	27.80	34.00	36.80	87.00	86.33	84.67	19.92	18.75	16.41
	GTTO [†]	71.11	79.75	81.05	29.20	35.40	37.20	87.33	86.00	87.00	22.66	17.97	21.48
	MargExKL [‡]	+1.59	+1.36	+1.52	+2.20	+2.20	+2.20	+2.33	+1.33	+1.67	+6.64	+1.17	+5.07
		70.81	79.34	80.67	31.40	35.80	38.40	86.33	85.50	86.33	23.05	21.48	22.27
LLaDA-1.5	top1_prob	71.11	80.89	81.35	27.20	34.40	36.40	85.67	88.67	85.33	18.75	21.88	19.14
	prob_margin	72.63	79.83	81.05	28.00	34.60	37.80	87.33	89.67	86.00	22.27	23.83	20.70
	GTTO [†]	74.07	81.81	82.25	29.60	36.20	39.20	88.67	89.67	87.33	23.83	24.22	22.27
	MargExKL [‡]	+2.96	+0.92	+0.90	+2.40	+1.80	+2.80	+3.00	+1.00	+2.00	+5.08	+2.34	+3.13
		73.39	81.90	82.11	31.60	37.40	39.40	87.67	89.33	87.00	25.39	25.00	26.17
		+2.28	+1.01	+0.76	+4.40	+3.00	+3.00	+2.00	+0.66	+1.67	+6.64	+3.12	+7.03

Table 3: Generation-length sensitivity of token-ordering policies (vote-answer accuracy). All gains are measured against top1_prob. [†] GTTO and [‡] MargExKL use tuned hyperparameters; additional sweep and cross-task results are reported in Appendix C and Appendix D.

First, the task-dependent split between GTTO and MargExKL suggests that the two policies capture different forms of temporal reliability. GTTO relies on discrete top-1 token agreement, so it is most useful when stable argmax predictions are a good proxy for reliable commitment. One possible explanation is that GSM8K and SVAMP, as arithmetic word-problem datasets, often have more constrained answer formats than MATH500 or Countdown. MargExKL, by contrast, uses a distribution-level KL proxy. This is more useful on MATH500 and Countdown, where the top-1 token may fluctuate even when the predictive distribution is gradually becoming more reliable. The strong MargExKL gains on MATH500 and Countdown therefore suggest that distribution-level stability can capture reliability signals missed by binary top-1 agreement.

Second, probability margin remains an essential component. The proposed methods do not replace margin; they build on it. GTTO reduces to margin-based ordering whenever the current margin is sufficiently high, and MargExKL preserves the margin term while discounting unstable distributions. This is important because margin captures current-step decisiveness, while temporal reliability captures cross-step consistency.

Appendix B further shows why gating is necessary. A naive temporal_margin rule that adds top-1 stability to every position performs worse than prob_margin on the GSM8K smoke test. This suggests that temporal stability is not universally helpful: when the current margin is already decisive, historical stability can perturb otherwise reliable choices. GTTO addresses this failure mode by activating the stability term only under local ambiguity, using temporal information as a selective correction rather than a global reweighting signal. Finally, to verify that the observed gains are not solely an artifact of TSCV aggregation, Appendix F reports final-step accuracy (without TSCV) alongside vote-answer accuracy. Both proposed policies improve over top1_prob under the final-step metric as well, confirming that token-ordering quality contributes independently of the aggregation mechanism.

Table 3 shows that the effect of temporal reliability depends on generation length. At $L = 128$, both GTTO and MargExKL improve over top1_prob on all tasks and both model checkpoints. At $L = 256$ and $L = 512$, however, the proposed methods do not uniformly dominate every baseline. One likely reason is that in LLaDA’s semi-autoregressive block decoding, committed tokens are never revised; longer sequences therefore provide a more complete reasoning chain as conditioning context by the time the answer region is decoded, which can reduce trajectory instability for simpler, more constrained tasks and thus lower the marginal value of temporal reordering. Another possible reason is that hyperparameters selected from $L = 128$ experiments may not be optimal for longer sequences, where the number of generated tokens, refinement dynamics, and answer trajectories differ.

The clearest long-length benefit appears on Countdown and MATH500, especially for MargExKL. These tasks involve more diverse or search-like reasoning trajectories, so distribution-level instability remains informative even when additional decoding budget is available. In contrast, on GSM8K and SVAMP, longer generation budgets partially reduce the gap between the proposed methods and

simpler baselines. This suggests that temporal reliability is most helpful when the denoising trajectory remains unstable, rather than when additional length alone is enough to stabilize the output.

The two temporal signals also differ in computational cost. GTTO only tracks run-length stability of the top-1 token and adds negligible overhead. MargExKL is more expensive because it retrieves previous-step probabilities to compute the KL penalty. As reported in Appendix A, this overhead remains modest in our setting, but it becomes more noticeable at longer generation lengths. Thus, MargExKL provides stronger gains on MATH500 and Countdown, while GTTO offers a cheaper token-level stability signal.

Finally, the exp-KL decay ablation shows that distributional stability alone is insufficient (full results in Appendix E). This variant ranks positions using only $\exp(-\gamma \widehat{KL})$, without the probability-margin term. It remains below `prob_margin` across all conditions and is generally weaker than `top1_prob`, with only minor exceptions on low-baseline Countdown settings. This failure indicates that a stable distribution is not necessarily a correct or decisive distribution. The result directly motivates the multiplicative form of MargExKL: KL is not used as a standalone confidence score, but only as a penalty on probability margin. MargExKL therefore preserves current-step decisiveness while discounting positions whose predictive distributions are unstable.

6 Conclusion

This paper investigates temporal reliability as a lightweight token-ordering signal in deterministic masked diffusion decoding. We revisit probability margin as a stronger local uncertainty baseline than raw top-1 confidence, and build two training-free policies on top of it. GTTO applies a gated stability term that activates only when the current margin falls below an ambiguity threshold, avoiding interference with already-decisive positions. MargExKL instead penalizes the margin by step-to-step KL divergence, discounting positions whose predictive distributions shift sharply across denoising steps. The two methods capture complementary notions of temporal reliability: GTTO tends to be more effective on arithmetic word problems with more constrained answer formats, while MargExKL is stronger on tasks involving more diverse or search-like reasoning trajectories.

At the main decoding setting, both methods improve over `top1_prob` across all four benchmarks and both model checkpoints. At longer generation lengths, the gains over `top1_prob` often remain positive, but their magnitude varies by task, model, and method. The ablation results further clarify why the proposed designs are needed: naive temporal-margin reweighting can hurt without a gate, and KL-only ranking fails without the probability-margin term. Taken together, these findings suggest that temporal information in DLMs can be useful not only for trajectory-level aggregation after generation, but also for token-level decisions that shape the denoising trajectory.

This work is limited in scope to the LLaDA model family and reasoning-focused benchmarks; generalization to other dLLM architectures or open-ended tasks remains an open question. A second limitation is that both GTTO and MargExKL introduce hyperparameters (λ , τ and γ , respectively) whose optimal values are task-dependent and currently selected via sweep. Future work could explore adaptive threshold mechanisms that infer ambiguity boundaries from the decoding trajectory itself, or learned gating functions that replace fixed hyperparameters with data-driven commitment criteria.

References

- [1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed H. Chi, Quoc V. Le, and Denny Zhou. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems*, volume 35, pages 24824–24837, 2022.
- [2] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023.
- [3] Jaeyeon Kim, Kulin Shah, Vasilis Kontonis, Sham Kakade, and Sitan Chen. Train for the worst, plan for the best: Understanding token ordering in masked diffusions. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *PMLR*, pages 30749–30768, 2025.

- [4] Wen Wang, Bozhen Fang, Chenchen Jing, Yongliang Shen, Yangyi Shen, Qiuyu Wang, Hao Ouyang, Hao Chen, and Chunhua Shen. Time is a feature: Exploiting temporal dynamics in diffusion language models. In *International Conference on Learning Representations*, 2026.
- [5] Pengxiang Li, Yefan Zhou, Dilxat Muhtar, Lu Yin, Shilin Yan, Li Shen, Soroush Vosoughi, and Shiwei Liu. Diffusion language models know the answer before decoding. In *International Conference on Learning Representations*, 2026.
- [6] Kecheng Chen, Ziru Liu, Xijia Tao, Hui Liu, Xinyu Fu, Suiyun Zhang, Dandan Tu, Lingpeng Kong, Rui Liu, and Haoliang Li. Beyond confidence: Adaptive and coherent decoding for diffusion language models. *arXiv preprint arXiv:2512.02044*, 2025.
- [7] Seo Hyun Kim, Sunwoo Hong, Hojung Jung, Youngrok Park, and Se-Young Yun. KLASS: KL-guided fast inference in masked diffusion models. In *Advances in Neural Information Processing Systems*, volume 38, pages 92267–92301, 2025.
- [8] Jacob Austin, Daniel D. Johnson, Jonathan Ho, Daniel Tarlow, and Rianne van den Berg. Structured denoising diffusion models in discrete state-spaces. In *Advances in Neural Information Processing Systems*, volume 34, pages 17981–17993, 2021.
- [9] Aaron Lou, Chenlin Meng, and Stefano Ermon. Discrete diffusion modeling by estimating the ratios of the data distribution. *arXiv preprint arXiv:2310.16834*, 2023.
- [10] Subham Sekhar Sahoo, Marianne Arriola, Yair Schiff, Aaron Gokaslan, Edgar Marroquin, Justin T Chiu, Alexander Rush, and Volodymyr Kuleshov. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems*, volume 37, pages 130136–130184, 2024.
- [11] Jiaxin Shi, Kehang Han, Zhe Wang, Arnaud Doucet, and Michalis K. Titsias. Simplified and generalized masked diffusion for discrete data. 37:103131–103167, 2024.
- [12] Shen Nie, Fengqi Zhu, Zebin You, Xiaolu Zhang, Jingyang Ou, Jun Hu, Jun Zhou, Yankai Lin, Ji-Rong Wen, and Chongxuan Li. Large language diffusion models. *arXiv preprint arXiv:2502.09992*, 2025.
- [13] Jiacheng Ye, Zhihui Xie, Lin Zheng, Jiahui Gao, Zirui Wu, Xin Jiang, Zhenguo Li, and Lingpeng Kong. Dream 7B: Diffusion large language models. *arXiv preprint arXiv:2508.15487*, 2025.

A Implementation Details

Hardware and distributed setup. All experiments are conducted on two NVIDIA H100 GPUs using PyTorch distributed data parallel (`torchrun`), with a per-process batch size of 4 and a global random seed of 42. No gradient computation is performed; all runs are inference-only.

Inference time. Wall-clock time scales superlinearly with generation length because both sequence length and diffusion steps double simultaneously. Under the `top1_prob` baseline, GSM8K (1,319 examples) requires approximately 8 minutes at $L = 128$ and 20 minutes at $L = 256$; Countdown (256 examples) requires approximately 1.5 and 4 minutes, respectively. The empirically observed scaling factor is approximately $2.5\times$ – $2.6\times$ from $L = 128$ to $L = 256$, and $2.8\times$ – $2.9\times$ from $L = 256$ to $L = 512$, somewhat below the theoretical $4\times$ due to attention kernel efficiency at longer sequences.

The per-configuration overhead of our proposed methods over `top1_prob` is negligible at $L = 128$ (under 21 seconds across all tasks) and grows modestly at longer lengths, reaching up to approximately 3 minutes for MargExKL at $L = 512$ on GSM8K. The additional cost of MargExKL relative to GTTO reflects the need to store the previous step’s predictive distribution so that previous-step probabilities for the selected top- M tokens can be retrieved, consistent with the $O(B \times |\mathcal{V}|)$ memory overhead noted in Section 3.4.

B Naive Temporal-Margin Smoke Test

To test whether token-level temporal stability can be used directly, we run a 64-example GSM8K smoke test comparing `prob_margin` against `temporal_margin`, defined as $\phi_i(t) = m_i(t) + \lambda s_i(t)$, for $\lambda \in \{0.05, 0.10, 0.20\}$. As shown in Table 4, every `temporal_margin` setting lowers both vote and final accuracy below `prob_margin`. This suggests that temporal stability is not reliable as an unconditional reweighting signal in this setting: when the current margin is already decisive, the stability term can re-rank otherwise reliable choices based on history that adds noise rather than information. This is the direct motivation for GTTO’s gated formulation.

Method	λ	Vote	Final
<code>top1_prob</code>	–	76.56	76.56
<code>prob_margin</code>	–	79.69	78.12
<code>temporal_margin</code>	0.05	75.00	73.44
<code>temporal_margin</code>	0.10	76.56	73.44
<code>temporal_margin</code>	0.20	75.00	71.88

Table 4: Naive temporal-margin smoke results on a 64-example GSM8K subset. Adding temporal stability unconditionally to the margin reduces both metrics relative to `prob_margin`, motivating the gated formulation in GTTO.

C GTTO Hyperparameter Selection and Cross-Task Transfer

We evaluate three competitive GTTO configurations across tasks, model checkpoints, and generation lengths in Table 5. The results show that no single (λ, τ) setting universally dominates. For example, (0.10, 0.15) is strong on several LLaDA-8B-Instruct settings, while (0.05, 0.12) is stronger for LLaDA-1.5 on GSM8K and MATH500. This motivates reporting tuned GTTO configurations in the main table, while using the appendix to expose the underlying cross-task sensitivity.

	Method	GSM8K			MATH500			SVAMP			Countdown		
		128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	top1_prob	69.52	78.39	79.53	27.00	33.20	35.00	85.00	84.67	85.33	16.02	16.80	16.41
	prob_margin	69.07	77.26	78.77	27.80	34.00	36.80	87.00	86.33	84.67	19.92	18.75	16.41
	GTTO ($\lambda=0.10, \tau=0.15$)	71.11	79.57	81.05	29.20	35.40	37.20	86.67	86.00	87.00	22.66	17.97	17.19
	GTTO ($\lambda=0.05, \tau=0.15$)	70.20	79.75	79.83	28.00	34.60	37.00	87.33	85.00	86.00	21.48	17.58	20.31
	GTTO ($\lambda=0.05, \tau=0.12$)	70.43	79.57	80.06	28.40	35.20	37.00	87.33	85.00	85.67	22.66	17.97	21.48
LLaDA-1.5	top1_prob	71.11	80.89	81.35	27.20	34.40	36.40	85.67	88.67	85.33	18.75	21.88	19.14
	prob_margin	72.63	79.83	81.05	28.00	34.60	37.80	87.33	89.67	86.00	22.27	23.83	20.70
	GTTO ($\lambda=0.10, \tau=0.15$)	72.71	81.81	82.03	28.80	35.40	39.20	88.67	89.67	87.33	23.05	24.22	21.09
	GTTO ($\lambda=0.05, \tau=0.15$)	73.16	81.23	81.50	29.20	35.60	39.20	87.67	89.33	86.33	23.05	23.05	21.88
	GTTO ($\lambda=0.05, \tau=0.12$)	74.07	81.65	82.25	29.60	36.20	39.20	87.67	89.33	87.00	23.83	23.44	22.27

Table 5: GTTO token-ordering results across generation lengths ($L \in \{128, 256, 512\}$) on LLaDA-8B-Instruct and LLaDA-1.5. Values are vote-answer accuracy (%). Bold indicates the best score among all methods shown for each model–task–length group; ties are bolded together.

D MargExKL Hyperparameter Selection and Cross-Task Transfer

Table 6 reports cross-task results for $\gamma \in \{0.3, 0.5\}$ across generation lengths and model checkpoints. In the main results, we report the better setting selected from this small sweep for each task and model. The table shows that the preferred γ is task-dependent: $\gamma = 0.5$ is generally stronger on MATH500, while $\gamma = 0.3$ is more competitive on Countdown, especially at longer generation lengths. This supports the view that the appropriate strength of KL penalization depends on the structure of the reasoning trajectory.

Model	Method	GSM8K			MATH500			SVAMP			Countdown		
		128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	top1_prob	69.52	78.39	79.53	27.00	33.20	35.00	85.00	84.67	85.33	16.02	16.80	16.41
	prob_margin	69.07	77.26	78.77	27.80	34.00	36.80	87.00	86.33	84.67	19.92	18.75	16.41
	MargExKL ($\gamma=0.5$)	70.81	78.88	80.67	31.40	35.80	37.40	86.33	85.50	86.33	23.05	19.14	16.41
	MargExKL ($\gamma=0.3$)	70.36	79.34	80.14	30.00	34.20	38.40	86.00	84.00	86.33	21.48	21.48	22.27
LLaDA-1.5	top1_prob	71.11	80.89	81.35	27.20	34.40	36.40	85.67	88.67	85.33	18.75	21.88	19.14
	prob_margin	72.63	79.83	81.05	28.00	34.60	37.80	87.33	89.67	86.00	22.27	23.83	20.70
	MargExKL ($\gamma=0.5$)	73.39	81.90	82.11	31.60	37.40	39.40	87.67	89.33	86.67	21.88	22.27	26.17
	MargExKL ($\gamma=0.3$)	73.34	81.18	82.03	30.40	36.00	39.00	87.33	89.00	87.00	25.39	25.00	25.78

Table 6: Cross-task transfer of selected MargExKL settings across generation lengths ($L \in \{128, 256, 512\}$) on LLaDA-8B-Instruct and LLaDA-1.5 (vote-answer accuracy, %). Bold indicates the best score among all methods shown for each model–task–length group; ties are bolded together. $\gamma = 0.5$ leads on MATH500 across nearly all conditions, while $\gamma = 0.3$ is more competitive on Countdown at longer generation lengths, consistent with the task-dependent split discussed in Section 5.

E exp-KL Decay Ablation: Full Results

Table 7 reports vote-answer accuracy for the exp-KL decay ablation across all three generation lengths, both model checkpoints, and $\gamma \in \{0.5, 1.0, 2.0\}$. This variant ranks positions using only distributional stability, without the probability-margin term. It falls far below prob_margin across all conditions and is generally much weaker than top1_prob, with only minor exceptions on low-baseline Countdown settings. These results confirm that distributional stability alone is not sufficient for reliable token ordering.

	Method	GSM8K			MATH500			SVAMP			Countdown		
		128	256	512	128	256	512	128	256	512	128	256	512
LLaDA-8B-Instruct	top1_prob	69.52	78.39	79.53	27.00	33.20	35.00	85.00	84.67	85.33	16.02	16.80	16.41
	prob_margin	69.07	77.26	78.77	27.80	34.00	36.80	87.00	86.33	84.67	19.92	18.75	16.41
	exp-KL decay $\gamma=0.5$	56.71	66.03	68.08	20.80	25.00	26.00	79.67	77.67	77.33	16.80	10.94	12.50
	exp-KL decay $\gamma=1.0$	57.09	66.34	68.46	21.20	25.00	26.40	79.33	78.67	78.00	17.19	11.33	12.89
	exp-KL decay $\gamma=2.0$	57.09	66.94	69.07	20.60	25.00	26.20	79.67	78.67	77.67	15.62	11.72	12.11
LLaDA-1.5	top1_prob	71.11	80.89	81.35	27.20	34.40	36.40	85.67	88.67	85.33	18.75	21.88	19.14
	prob_margin	72.63	79.83	81.05	28.00	34.60	37.80	87.33	89.67	86.00	22.27	23.83	20.70
	exp-KL decay $\gamma=0.5$	60.42	67.85	68.99	24.00	28.00	28.80	80.00	76.67	76.00	17.19	14.06	14.45
	exp-KL decay $\gamma=1.0$	60.50	67.55	68.61	23.80	28.40	29.20	80.67	76.67	76.33	17.19	13.67	14.06
	exp-KL decay $\gamma=2.0$	60.35	67.48	68.61	24.20	28.80	29.40	80.67	76.33	76.33	17.19	13.28	13.67

Table 7: exp-KL decay ablation results across generation lengths ($L \in \{128, 256, 512\}$) on LLaDA-8B-Instruct and LLaDA-1.5. Values are vote-answer accuracy (%). Baselines (top1_prob, prob_margin) are reproduced for reference. Bold indicates the best score among all methods shown for each model-task-length group. The KL-only variants fall far below prob_margin across all conditions and are generally weaker than top1_prob, showing that distributional stability without a margin term is insufficient for reliable ordering.

F TSCV Ablation: Vote vs. Final-Step Accuracy

To assess the independent contribution of token-ordering policies from the Temporal Self-Consistency Voting (TSCV) aggregation mechanism, we compare two evaluation modes for each policy at generation length $L = 128$:

- **Vote** (with TSCV): the exponentially weighted majority-vote answer across all intermediate denoising steps, using $\alpha = 5.0$.
- **Final** (without TSCV): the answer predicted at the last denoising step only, equivalent to standard single-pass decoding accuracy.

Tables 8 and 9 report both metrics for LLaDA-8B-Instruct and LLaDA-1.5, respectively. The results show that the proposed policies improve over top1_prob under both evaluation modes, indicating that the gains are not solely attributable to TSCV aggregation. The results suggest that the proposed ordering policies improve not only the TSCV-aggregated vote accuracy but also the final-step prediction in most settings. This indicates that the gains are not solely due to the aggregation mechanism, although the size of the Vote-Final gap can vary across tasks and policies.

Method	GSM8K		MATH500		SVAMP		Countdown	
	Vote	Final	Vote	Final	Vote	Final	Vote	Final
top1_prob	69.52	67.93	27.00	26.80	85.00	83.67	16.02	14.06
prob_margin	69.07	67.63	27.80	27.80	87.00	86.00	19.92	18.36
GTTO [†]	71.11	69.67	29.20	28.40	87.33	86.33	22.66	20.31
MargExKL [‡]	70.81	69.07	31.40	30.60	86.33	84.67	23.05	20.70

Table 8: Vote-answer accuracy (with TSCV) vs. final-step accuracy (without TSCV) on LLaDA-8B-Instruct at $L = 128$. [†] GTTO and [‡] MargExKL use the tuned configurations reported in Table 2 for each task. Gains from token-ordering policies persist under both evaluation modes, suggesting that improvements are not solely attributable to TSCV.

Method	GSM8K		MATH500		SVAMP		Countdown	
	Vote	Final	Vote	Final	Vote	Final	Vote	Final
top1_prob	71.11	70.13	27.20	27.00	85.67	85.00	18.75	16.80
prob_margin	72.63	71.57	28.00	27.40	87.33	86.00	22.27	19.53
GTTO [†]	74.07	72.63	29.60	28.80	88.67	87.33	23.83	19.14
MargExKL [‡]	73.39	72.40	31.60	30.80	87.67	86.63	25.39	22.66

Table 9: Vote-answer accuracy (with TSCV) vs. final-step accuracy (without TSCV) on LLaDA-1.5 at $L = 128$. [†] GTTO and [‡] MargExKL use the tuned configurations reported in Table 2 for each task. Both proposed policies improve over top1_prob in the Final column in this setting, suggesting that ordering-level gains are present independently of TSCV aggregation.